Problem Statement: Multi-Modal Real-Time Dashboard Web App (Full-Stack Assessment)

Overview

You are required to build a production-style web application that demonstrates real-time data ingestion, periodic API polling, state/session management, and a polished multi-page UI with consistent theming and animations. The application will be a dashboard for a selected domain (any domain is acceptable). Your goal is to create a working system with both frontend and backend components.
The project will be evaluated against functional correctness, UI quality, code structure, and completeness of required features.

Tech Stack

Use any reasonable combination of:
• JavaScript/TypeScript
• React
• CSS (or a CSS preprocessor)
• UI Design Framework (e.g., Tailwind CSS, Material UI, Ant Design, Chakra UI, etc.)
• Animation Framework (e.g., Framer Motion, React Spring, GSAP, etc.)
• State Management: Redux or Zustand
You may use additional libraries as needed, as long as all requirements below are met.

Domain

Choose any domain you like (the domain must be coherent across the UI labels, charts, metrics, tables, and API payloads).
Examples (choose one):
• Smart warehouse sensor monitoring
• Fleet/vehicle telemetry dashboard
• IoT environment control system
• Stock/crypto market simulator dashboard
• Campus event tracker / traffic monitoring simulator
• Fitness metrics & habit tracker
• E-commerce / sales & conversion analytics simulator
• Any other real dashboard-like domain

Application Requirements

1) Live Real-Time Data Ingestion in UI Screen

Create at least one dashboard page that displays live-updating dummy data streamed from the backend.
• The UI must update without manual refresh.
• Display at least 3 different real-time metrics.
• Include a visible “LIVE” indicator and show the timestamp of the last received update.
• Live metrics should visibly change over time (e.g., fluctuating numbers, changing statuses, or incoming events).
Backend expectation: Provide a streaming or push mechanism such as WebSocket or Server-Sent Events (SSE), or long-polling or any equivalent streaming approach. The frontend must consume and ingest updates into UI state.

2) Periodically Fetching of API and Data Ingestion in UI Screen

In addition to the live stream, implement a section/page that periodically calls backend APIs.
• Fetch data every X seconds (example range: 5–15 seconds).

• Show “Last updated” timestamp for periodic data.
• Periodic results must differ logically from the live feed.
Example: live stream can show last N seconds of raw telemetry/events; periodic fetch can show aggregated stats (averages/totals), trends, summaries, leaderboards, or derived alerts.
Backend expectation: Provide at least two API endpoints for polling aggregated/derived data, for example GET /api/dashboard/summary and GET /api/dashboard/alerts (or equivalent endpoints, as long as the UI uses them periodically).

3) Color Combination Properly Maintained (Theming Consistency)

Maintain a consistent and professional theme across the application.
• Define a primary color palette, secondary/accent colors, background, and text colors.
• Ensure cards, buttons, charts, inputs, and key UI elements follow the same theme rules.
• Ensure readability and contrast.
• If you implement dark mode/light mode, both must remain consistent.
At minimum, consistent theming must be applied across:
• Login / Entry page
• Dashboard pages
• Settings / Session-related page

4) Session Management Handling (Frontend)

Implement frontend session management.
• Provide a login flow (dummy credentials are acceptable).
• Protect at least one route (dashboard area) so that unauthenticated users are redirected to login.
• Handle expired/invalid session (e.g., clear session and redirect).
• Handle logout functionality.
• Handle safe state transitions when session expires during API calls.
Backend expectation: Include authentication/session endpoints and protect API routes required by the dashboard (at minimum, one endpoint).

5) At Least 5–6 Pages (Multi-Page App)

Build a multi-page experience using routing. The app must include at least 5 or 6 distinct pages.
Recommended page structure (you may modify):
• Public Entry Page (Login / Welcome)
• Dashboard Overview (Live stream + key widgets)
• Analytics / Trends page (charts based on periodic APIs or derived data)
• Alerts / Events page (event list, filtering, details)
• Settings / Preferences page (theme, refresh interval, live pause/resume settings)
• Profile / Session Activity page (optional but recommended)
Mandatory details: Each page must contain meaningful UI connected to the domain data. Navigation between pages should be smooth and intuitive.

6) Project of Any Domain

The project can be any domain, but it must be coherent and consistent across the app.
• Domain must be visible in headings and UI labels.
• Metric naming, entities, and meaning must remain consistent.
• Both live stream content and periodic API content must align with the domain narrative.

7) Multi-Modality Interactivity in UI

Your UI must include multi-modality interactivity (at least two different interaction types).

Examples (pick at least two; more is better):
• Visual: animated charts, micro-animations, hover states, animated progress indicators, expand/collapse cards
• Form: filtering, search, sorting controls affecting displayed data
• Real-time: pause/resume live updates, threshold slider affecting alerts
• Modal/Drawer: click a card/table row to open a details modal
• Optional: drag-to-reorder dashboard widgets
Mandatory details:
• Include at least one user control that changes what is rendered.
• Include at least one interaction that triggers a visible UI response (modal, drawer, expanded panel, toast, or live mode toggle).

Dummy Live Data + Backend Generation (Explicit Requirement)

Live Dummy Data Generator

You must generate dummy live data in the backend that feeds the realtime UI.
• Dummy live data must include at least 3 numeric metrics that fluctuate over time.
• At least one categorical or status field (e.g., OK/WARN/CRITICAL).
• At least one event type or event label (e.g., UPDATE/ALERT/RECOVERY).
• Timestamp.
Backend must continuously generate events and send them to the frontend.

Periodic Dummy Data for Polling

You must also generate dummy data for periodic fetching endpoints.
• Periodic endpoints should return structured aggregated/derived content such as averages/totals for a time window.
• Trend deltas vs previous window.
• A derived alerts list based on thresholds.
• Periodic results must be meaningfully different from the raw live stream payloads.